科目：データベースの基礎

第2回：リレーショナルデータモデル：構造記述

講師：鈴木源吾

前回の復習

- データベースとは？
  - 意味1：多くのデータを集めたもの
  - 意味2：多くのデータを集めるコンピュータを利用したシステム（仕組み）
  - 意味3：意味2のために利用する，データを管理するためのソフトウェア
- 意味3のソフトウェアをデータベース管理システム（DBMS）と呼ぶ
- 狭義の定義 **「複数の応用目的での共有を意図して組織的かつ永続的に格納されたデータ群」**
  - 4つのキーワード：永続的・共有・組織的・複数応用

第1回演習の解説

- 第1回ポイント資料の裏面（演習1・演習2・確認問題）の正解を解説する
- 解説は，演習の締切（講義開始30分後）のあとに行う
- 解答例は電子的には配布しない．**自分の解答に転記すること**（解説中の撮影は不可）
  - 講義中に自分の解答へ書き足す・訂正するのはよい．そのまま試験に持ち込める
- 提出がまだの人は，WebClass の「【締切後】第1回 演習」から提出する（0.7点で評価）

今回のトピック

1. リレーショナルデータベースについて
2. リレーショナルデータモデルの構造
3. 第1正規形
4. 今回の演習と小テストの「型」

教科書：第2章 p.15〜24

Topic 1

リレーショナルデータベースについて

リレーショナルデータベースとは

「リレーショナルデータモデル」に従って作られたデータベースのこと

- リレーショナルデータモデルは，「表形式」のリレーション（関係）を基礎とする
- 問合せ言語として，標準化された**SQL言語**が使えるのが大きな特徴
  - Structured Query Language
  - 基本はシンプルだが，非常に多機能に拡張された，強力な言語

リレーショナルデータベースを作成するためのデータベース管理システムをリレーショナルデータベース管理システム（RDBMS）と呼ぶ

リレーショナルデータベースの歴史

リレーショナルデータベースは、現在ではデータベースの代名詞となっている

- 1970年：Codd博士が抽象的なリレーショナルデータモデルを提案
- 1980年代：SQL言語の開発、先進製品の開発
- 1990年代：SQL言語標準化と普及(Oracleなど)
- 2000年代〜現在：オープンソースのDBMSの普及（PostgreSQL・MySQL・SQLite等）

歴史的には，リレーショナルデータベースは，最初は抽象的なモデルに従っているために性能が低いとされていたが，実用的に改善され現代に至る

RDBMSの例

商用

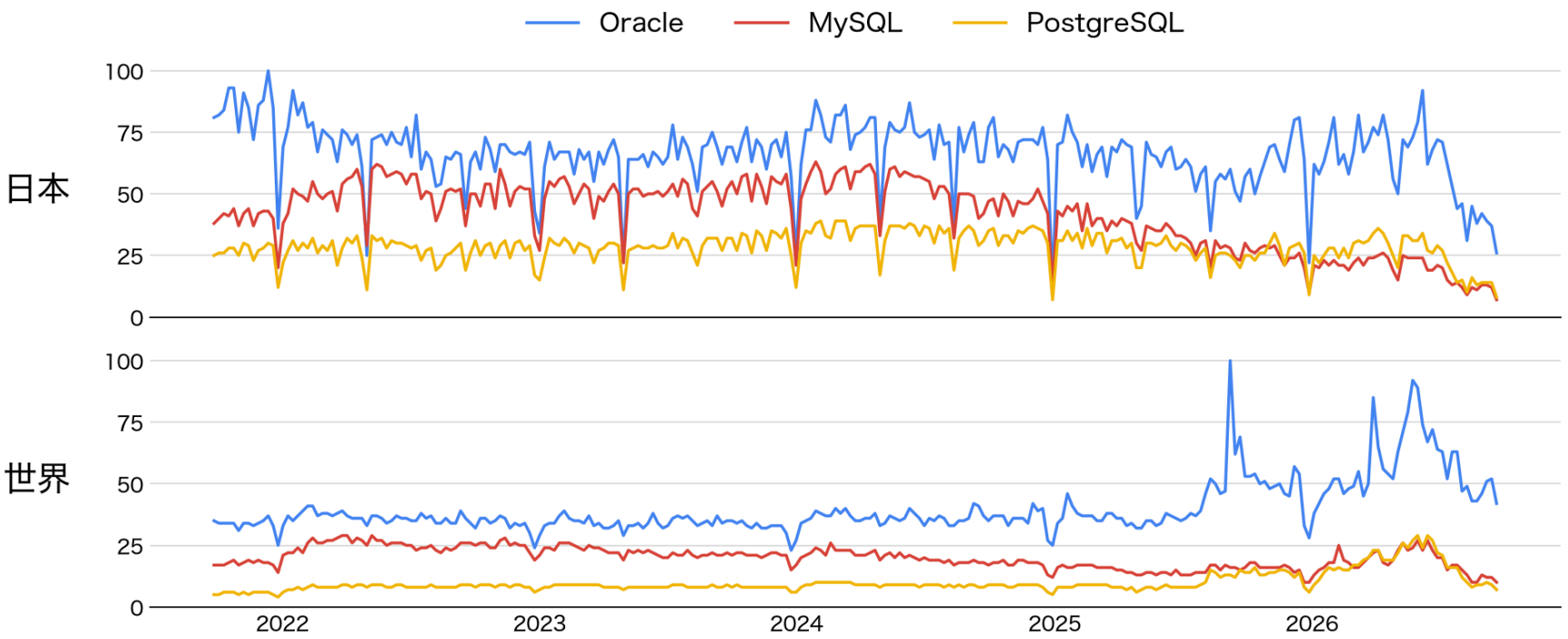
- Oracle: 商用で，最も普及しているRDBMS

OSS（Open Source Software）

- PostgreSQL: 多機能．データ分析には比較的強い．日本ではシェア高め
- MySQL: OSSではシェアトップ．クラスタリングに強い（複数のDBを合わせて容量や性能をあげる）
- SQLite: 軽量．RDBMS入門によく使われる

Googleトレンドで見てみる：MySQL vs. PostgreSQL

検索人気度の推移（5年間）：日本でも世界でも，PostgreSQL が MySQL に並び，追い越しつつある



出典：Google トレンド（検索人気度，2026年9月20日の週までの5年間）

リレーショナルデータベースの限界とNoSQL

2010年代に入って，リレーショナルデータベースの限界が指摘されるようになり，NoSQLデータベースと呼ばれる動きが広がった（NoSQL自体に技術的な定義はない）．その求められた要因は，

- リレーショナルデータベースを超えるスケーラビリティ（極めて巨大なデータセットや，書き込みのスループット等）
- リレーショナルデータモデルではうまくサポートされない特殊なクエリ操作
- リレーショナルデータモデルの制約を超えた，もっと動的で表現力に富むデータモデル

→　**ビッグデータを扱う上では重要なトピック．第14回で触れる**

Topic 2

リレーショナルデータモデルの構造

トピックのキーワード

- リレーション・タプル・属性
- リレーションスキーマ・次数・ドメイン

リレーション

- リレーショナルデータモデルの基礎はリレーション（関係）
- リレーションは2次元の「表」である．以下の用語の区別をする
  - リレーショナルデータモデルにおける表→リレーション
  - SQLにおける表（PostgreSQL上の表など）→テーブル
- リレーションには名前がある．リレーション名と呼ぶ

社員 — **リレーション名**

社員番号	社員名	給与	所属	健保番号
0650	山田太郎	50	K55	80596
1508	鈴木花子	40	K41	81403
0231	田中桃子	60	K41	80201
2034	佐藤一郎	40	K55	81998

リレーションの構成要素

- リレーションは，「行」の集まりである
- この「行」を**タプル**（または，**レコード**）と呼ぶ
- タプルが何に関する値を持つかを表すもの，つまり，「表の列」を**属性**（または，**カラム**，列）と呼ぶ



社員番号	社員名	給与	所属	健保番号
0650	山田太郎	50	K55	80596
1508	鈴木花子	40	K41	81403
0231	田中桃子	60	K41	80201
2034	佐藤一郎	40	K55	81998

リレーションスキーマ

- タプルの属性の「見出し」を**属性名**（または**カラム名**）と呼ぶ
- 表に含まれるすべての属性名をまとめたものを**リレーションスキーマ**と呼ぶ
- リレーションスキーマは，リレーションの「構造」を表している
  - 属性の数を「**次数**（degree）」と呼ぶ
- リレーションスキーマは，メタデータ（データを表すためのデータ）の一つ



リレーショナルデータモデルの決まりごと (1/3)

- 単に表という以上に決まりごとがある

ルール1：タプルの順序に意味はない

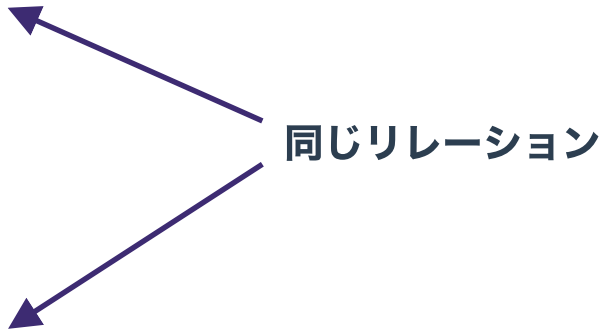
- リレーショナルデータベースの検索結果は（特に指定しなければ）順序は保証されない

社員

社員番号	社員名	給与	所属	健保番号
0650	山田太郎	50	K55	80596
1508	鈴木花子	40	K41	81403

社員

社員番号	社員名	給与	所属	健保番号
1508	鈴木花子	40	K41	81403
0650	山田太郎	50	K55	80596



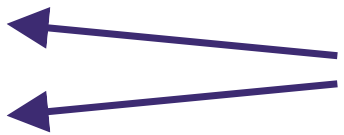
リレーショナルデータモデルの決まりごと (2/3)

ルール2：値が一致すれば，それは同じもの

- すべての属性の値が一致すれば，その2つのタプルは同じもの
- つまり，タプルは，数学と同じように，集合の要素と解釈されている
- （若干ややこしいが）しかし，これは実際のRDBMSでは区別される．
- このルールは，「形式的な」リレーショナルデータモデルでの決まりごと

社員

社員番号	社員名	給与	所属	健保番号
0650	山田太郎	50	K55	80596
0650	山田太郎	50	K55	80596



区別する手段なし

リレーショナルデータモデルの決まりごと (3/3)

ルール3：値に入るのは，構造を持たない「単項目」のデータに限る

- 配列やリレーションをリレーションの中に入れることはできない
  - 通常は数値や文字列
- （若干ややこしいが）実際の多くのDBMSではこのルールを拡張している
  - 「オブジェクト」を入れることができる
- このルールは，「形式的な」リレーショナルデータモデルでの決まりごと

リレーショナルデータモデルを形式的に定義する

- 今まで述べた「決まりごと」は実はもともとは，先に形式的（フォーマル）・数学的な定義があった．それをことばで述べたもの
- いきなり，数学的定義がでてくるととまどうと考えたので文章で述べた
- リレーショナルデータモデルは集合を使って定義される

リレーションの定義

定義：$D_1, D_2, ..., D_n$を有限個のドメインとするとき，$D_1, D_2, ..., D_n$上のリレーションRとは，直積$D_1 \times D_2 \cdots \times D_n$の任意の有限部分集合を言う

- ドメイン（定義域）：属性のとりうる値を集めた集合
  - 例：文字列全体，数値全体，真偽値
  - 例：最大長10の文字列全体
  - 例：人名の集合，給与の集合，年齢の集合
  - 無限集合でもかまわない
- 前に述べた「決まりごと」とは，「リレーションは集合である（集合を基礎とする）」ことを言い換えているにすぎない

定義に従ったリレーションの例

- 下のリレーション「社員」は，次のドメインの直積の部分集合
  - D_1：社員番号（4桁の数字の文字列）の集合，D_2：人名の集合，$D_3 = \{x : x\text{は自然数}\}$（給与）
  - D_4：部署を表す文字列の集合，D_5：健保番号（5桁の数字の文字列）の集合
- タプル（要素）の例：('0650', '山田太郎', 50, 'K55', '80596')

社員 — リレーション名

社員番号	社員名	給与	所属	健保番号
0650	山田太郎	50	K55	80596
1508	鈴木花子	40	K41	81403
0231	田中桃子	60	K41	80201
2034	佐藤一郎	40	K55	81998

リレーションの定義についての補足

- 例の「社員」は教科書 図3.1（p.26）のリレーション．次回のキーの説明でも使う
- 教科書では，さらに「ドメイン関数を用いたリレーションの定義法」が述べられているが，わかりにくい割には，それほど役に立たないので，説明はしない
- 属性名とドメインが対応すること，が理解できていればよい
  - 属性Aのドメインは Dである，という言い方をする
- 属性名の集まりをリレーションスキーマとすることも理解しておくこと

Topic 3

第1正規形

トピックのキーワード

- 第1正規形・入れ子型リレーション
- 正規化

第1正規形とは

まず，議論を徒に複雑にしないために，リレーションには以下の制約を置く

定義：リレーションの各属性のドメインが「シンプル」であるとき，そのリレーションを第1正規形であるという

- 「シンプル」とは，値に組（タプル）・集合・配列のような「構造」を含まないことを言う
- 文字列や数値は，シンプルなドメインである
- 第1正規形ではない，値に「構造」を含むようなものを，入れ子型リレーション（ネステッド・リレーション）と呼ぶ．教科書では「非第1正規形」
- これは主に正規化理論のための制約であり，実用的な観点で現代のDBMSでは，構造を含むドメインを扱えるようになっている

第1正規形ではない例

下の2つの社員リレーションは，第1正規形ではない

- 左の社員の属性「社員名」の値は，姓と名の組であるから第1正規形ではない
- 右の社員の属性「趣味」の値は，趣味の値の集合であるから第1正規形ではない

社員

社員番号	社員名	…
0650	(山田, 太郎)	…
1508	(鈴木, 花子)	…

値が組

社員

社員番号	社員名	趣味	…
0650	山田太郎	{読書, ブログ, 筋トレ}	…
1508	鈴木花子	{料理, カラオケ}	…

値が集合

第1正規形にする

第1正規形ではない左の例（値が組）を第1正規形に直してみよう
正規形に直すことを，「**正規化**」と呼ぶ

- 姓と名をそれぞれ別の属性にすればよい（教科書 図2.7）

社員

社員番号	社員名	…
0650	(山田, 太郎)	…
1508	(鈴木, 花子)	…

正規化 →

社員

社員番号	社員名_姓	社員名_名	…
0650	山田	太郎	…
1508	鈴木	花子	…

第1正規形にする

第1正規形ではない右の例（値が集合）を第1正規形に直してみよう

- 趣味には一つの値（シンプル）しかいれることができない
- よって，趣味ごとに別のタプルを作成することにより正規化する

社員

社員番号	社員名	趣味
0650	山田太郎	{読書, ブログ, 筋トレ}
1508	鈴木花子	{料理, カラオケ}

正規化



社員

社員番号	社員名	趣味
0650	山田太郎	読書
0650	山田太郎	ブログ
0650	山田太郎	筋トレ
1508	鈴木花子	料理
1508	鈴木花子	カラオケ

第1正規形ではない例

さらに，もう一つ典型的な例をあげよう

- ネットショッピングの注文をデータベース化することを考えよう
- 一つの注文には，以下の情報が入る
  - 注文した人の名前
  - 注文した日付
  - 注文した商品と個数の組，が商品ごとに複数個

第1正規形ではない例

素朴にはこんな構造のデータにしたくなる

- 注文1つに対して，商品のタプルが繰り返されるような構造
- 古いネットワーク型データベースでは，これに近い構造で管理していた（リピーティンググループと呼ばれる）
- これは，属性の値がタプルの集合という構造を持っているから第1正規形ではない

注文

注文番号	名前	日付	注文商品（商品名, 個数）
1	鈴木	11/7	{(iPad, 1), (ケーブル, 1)}
2	佐藤	11/4	{(MacBook, 3)}
3	鈴木	11/8	{(Apple Watch, 1), (バンド, 2), (充電器, 1)}

第1正規形にしてみよう：よくない例

次のような構造にしがちである（世の中にこんな構造のExcelファイルがいっぱいある）．これは確かに第1正規形であるが，以下の点などで好ましい構造とは言えない

- 同時に注文する商品の種類が増えると，属性を加えなくてはいけない
  - 多めに属性を作っておく案もあるが，属性が増えて扱いづらい
- （できないことはないが）一つの注文における商品の数が数えにくい
  - NULLである属性を探す必要がある→SQLではやりづらい

注文

注文番号	名前	日付	商品名1	個数1	商品名2	個数2	商品名3	個数3
1	鈴木	11/7	iPad	1	ケーブル	1		
2	佐藤	11/4	MacBook	3				
3	鈴木	11/8	Apple Watch	1	バンド	2	充電器	1

第1正規形にしてみよう：よりよい設計

- 社員の趣味の例と同じく，横にのばすのではなく，縦にタプルで商品を分ける
- 商品が増えてもタプルを追加すればよく，注文ごとの商品の数もSQLで容易に取得できる
- ただし，1タプルが1注文の1商品を表すため，主キーが変わっていることに気を付けること（第2正規形への正規化が必要になる場合がある．第7回）

注文

注文番号	名前	日付	商品名	個数
1	鈴木	11/7	iPad	1
1	鈴木	11/7	ケーブル	1
2	佐藤	11/4	MacBook	3
3	鈴木	11/8	Apple Watch	1
3	鈴木	11/8	バンド	2
3	鈴木	11/8	充電器	1

Topic 4

今回の演習と小テストの「型」

第1正規形への正規化のまとめ

構造	例	正規化の方法
組	社員名 ＝ (山田, 太郎)	組の要素を別々の**属性**に分ける
集合	趣味 ＝ {読書, ブログ, 筋トレ}	要素ごとに別の**タプル**に分ける．他の属性の値はコピーする
組の集合	注文商品 ＝ {(iPad, 1), (ケーブル, 1)}	上の2つを両方行う

- 「商品1，個数1，商品2，個数2，…」と横にのばすのは，よくない例

今回の小テストの「型」

- **型A：説明 → 用語**（第1回と同じ）
  - 例）リレーショナルデータモデルの「リレーション」にあたる，SQLでの表の呼び方 → テーブル
- **型C：第1正規形への正規化**　リレーションスキーマ → タプルの順で書く
  - 例）注文 (注文番号, 名前, 注文商品) で，注文商品が {(iPad, 1), (ケーブル, 1)} のような組の集合
  - 答）注文 (注文番号, 名前, 商品名, 個数)
    { (1, 鈴木, iPad, 1), (1, 鈴木, ケーブル, 1), (2, 佐藤, MacBook, 3) }
- 手順：構造を持つ属性を見つける → 組は属性に分ける → 集合はタプルに分ける → スキーマ・タプルの順で書く

第2回演習

- ポイント資料の裏面を，表面の例にならって自分の手で解く
  - 演習1：型Aの類題（6題）
  - 演習2：型Cの類題（3題）：値が組・集合・組の集合
  - 確認問題（2題）
- 提出：表面・裏面をそれぞれ撮影して，WebClass「第2回 演習」へ
  - 締切：**次回（10/14）の講義開始30分後**
- 次回の予習：教科書 第3章 p.25〜33（リレーショナルデータモデル：意味記述）